Programming Fundamentals

Week 7 — Wednesday: PC Troubleshooter and JSON

Programming Fundamentals Team

SETU

2026-10-28

From Files to a Knowledge Base

Learning goals: load structured data; represent troubleshooting knowledge in JSON; separate rules from code; save a session history; test missing-file cases.

Monday Recap: What Survives?

Predict:

```
with open("settings.json", "w") as file:
    json.dump({"theme": "dark"}, file)
```

After Python closes, where is the theme stored?

Monday Recap: What Survives?

Predict:

```
with open("settings.json", "w") as file:
    json.dump({"theme": "dark"}, file)
```

After Python closes, where is the theme stored?

In `settings.json`, not in a Python variable.

Our Ongoing Example: PC Troubleshooter

Previously we asked questions and used selection to suggest solutions.

Today: move some troubleshooting knowledge into a JSON file.

> The Python program decides **how** to process answers. The file supplies **what** questions and recommendations to use.

Why Store Rules in a File?

Suppose we want to add a new recommendation.

Hard-coded approach: edit the Python `if` statements.

Data-driven approach: add a record to the knowledge file.

which approach is easier for a non-programmer to maintain?

Design the Knowledge First

We will begin with two yes/no questions:

1. Does the computer turn on? (`power_on`)
2. Is the display showing an image? (`display_on`)

Possible recommendations:

- Check the power connection
- Check the monitor and cable
- Try a restart or further checks

Our Knowledge File: JSON

```
{
  "questions": [
    {"key": "power_on", "text": "Does it turn on?"},
    {"key": "display_on", "text": "Is there an image?"}
  ]
}
```

A **key** lets our program recognise each answer. The **text** is what the user sees.

Add Rules to the Same File

```
"rules": [
  {"when": {"power_on": "no"},
   "advice": "Check the power cable and socket."},
  {"when": {"power_on": "yes", "display_on": "no"},
   "advice": "Check the monitor connection."}
]
```

The first matching rule supplies the advice.

Pseudocode: Load the Knowledge

```
OPEN the knowledge file
READ the JSON data
STORE the questions and rules
ASK the user each question
COMPARE answers with each rule
DISPLAY advice for the first matching rule
```

Which steps belong in separate functions?

Step 1: Load the Knowledge

```
import json

def load_knowledge(filename):
    with open(filename, "r", encoding="utf-8") as file:
        return json.load(file)

knowledge = load_knowledge("knowledge.json")
```

What type of value does `load_knowledge()` return?

Step 2: Ask the Questions

```
def ask_questions(questions):
    answers = {}
    for question in questions:
        key = question["key"]
        answer = input(question["text"] + " (yes/no): ")
        answers[key] = answer.strip().lower()
    return answers
```

We use a **dictionary** to connect question keys to answers.

Step 3: Match a Rule

```
def matches(rule, answers):
    for key, expected in rule["when"].items():
        if answers.get(key) != expected:
            return False
    return True
```

Every condition in the rule must match.

Predict: what happens when the first condition fails?

Step 4: Choose Advice

```
def recommend(rules, answers):
    for rule in rules:
        if matches(rule, answers):
            return rule["advice"]
    return "No matching rule. Ask for further help."
```

Important: rule order matters because we return the first match.

Live Build: Put It Together

```
knowledge = load_knowledge("knowledge.json")
answers = ask_questions(knowledge["questions"])
advice = recommend(knowledge["rules"], answers)
print("Recommendation:", advice)
```

Try no/no, yes/no, and yes/yes.

Improve the Conversation

Our first version accepts anything, including `maybe`.

Adapt the question function to repeat until the user enters `yes` or `no`.

> Hint: use a `while` loop.

Save a Session History

```
from datetime import datetime

record = {"time": datetime.now().isoformat(timespec="seconds"),
          "answers": answers, "advice": advice}
```

We can store multiple records in a JSON **list**.

Why might a support technician want this history?

Save the Updated History

```
history = []
try:
    with open("history.json", "r") as f:
        history = json.load(f)
except FileNotFoundError:
    pass
history.append(record)
with open("history.json", "w") as f:
    json.dump(history, f, indent=4)
```

We load old records **before** saving the new list.

Challenge: Add a New Rule

Add a third question and a new rule in `knowledge.json`.

Suggested question: **Is the power cable connected?**

- Decide its `key`
- Write the new rule conditions
- Put the **more specific** rule before the general power rule
- Test which recommendation appears

Testing and File Problems

Try these tests:

- The knowledge file is missing
- The JSON contains a missing comma
- No rule matches the answers
- The user types `YES` instead of `yes`

These reveal why validation, sensible defaults and clear error messages matter.

Assignment 2 Connection

Your expert system can separate:

Knowledge — questions, rules, advice in JSON

Program logic — functions that load, ask, compare and explain

Persistence — saved records that remain after closing the program

Later we will explore how classes and objects can organise this further.

Exit Ticket and Next Steps

Before you finish, explain in your own words:

1. Why is JSON useful for an expert system?
2. Why do we load before we save a history?
3. What is one limitation of the first-match rule approach?